

Datenstruktur `HashMap` in Java

Kriterium	Beschreibung	Wann zu verwenden?
Schlüssel-Wert-Paar	Eine <code>HashMap</code> speichert Daten in Form von Schlüssel-Wert-Paaren, wobei der Schlüssel eindeutig sein muss.	Wenn eine Zuordnung von eindeutigen Schlüsseln zu Werten erforderlich ist, z.B. für eine schnelle Suche oder Abbildung.
Schneller Zugriff (O(1))	Der Zugriff auf Werte über einen Schlüssel ist im Allgemeinen sehr schnell (O(1) im Durchschnitt).	Wenn schneller Zugriff auf Werte basierend auf einem Schlüssel erforderlich ist.
Keine Reihenfolge	Eine <code>HashMap</code> garantiert keine Reihenfolge der Elemente. Die Reihenfolge der Einfügungen ist nicht beibehalten.	Wenn die Reihenfolge der Elemente nicht wichtig ist und nur die Zuordnung von Schlüsseln zu Werten benötigt wird.
Null-Werte	Eine <code>HashMap</code> erlaubt es, <code>null</code> als Wert zu speichern, jedoch nur ein <code>null</code> als Schlüssel.	Wenn <code>null</code> -Werte als gültige Daten akzeptiert werden oder <code>null</code> als Platzhalter für nicht zugewiesene Werte benötigt wird.
Schnelles Hinzufügen und Entfernen	Das Hinzufügen und Entfernen von Elementen erfolgt in durchschnittlicher O(1)-Zeitkomplexität.	Wenn häufige Einfügungen und Löschungen auf Basis von Schlüsseln erforderlich sind und Performance wichtig ist.
Verwaltung von Duplikaten	Ein <code>HashMap</code> -Schlüssel kann nur einmal vorkommen. Wenn ein neuer Wert für einen existierenden Schlüssel eingefügt wird, wird der alte Wert überschrieben.	Wenn es nur eine Instanz jedes Schlüssels gibt und die Werte dynamisch aktualisiert werden können.
Daten suchen und filtern	Durch die schnelle Suche nach Werten über den Schlüssel ist das Filtern von Daten effizient möglich.	Wenn schnell nach bestimmten Werten basierend auf dem Schlüssel gefiltert werden muss.
Speicherverbrauch	Eine <code>HashMap</code> benötigt mehr Speicher als eine einfache Array- oder List-basierte Struktur, da sie zusätzliche Daten zur Verwaltung der Schlüssel-Wert-Paare speichert.	Wenn der Speicherverbrauch nicht kritisch ist und die Leistung und Flexibilität der <code>HashMap</code> wichtiger sind.

Kriterium	Beschreibung	Wann zu verwenden?
Thread-Sicherheit	<code>HashMap</code> ist nicht threadsicher. Für Multithreading-Anwendungen sollte entweder eine <code>ConcurrentHashMap</code> oder explizite Synchronisierung verwendet werden.	Wenn keine Thread-Sicherheit erforderlich ist oder Synchronisierung manuell hinzugefügt werden kann.
Unterschiede zu <code>Hashtable</code>	Im Gegensatz zur <code>Hashtable</code> erlaubt die <code>HashMap</code> <code>null</code> als Wert und ist nicht synchronisiert.	Wenn Synchronisation nicht erforderlich ist und die Verwendung von <code>null</code> als Wert akzeptabel ist.
Leistung	Die <code>HashMap</code> bietet durchschnittlich eine sehr gute Leistung für Einfügungen, Löschungen und Suchen ($O(1)$ im Durchschnitt).	Wenn eine schnelle und performante Datenstruktur benötigt wird, insbesondere für große Datenmengen.
Verwendung von <code>equals</code> und <code>hashCode</code>	Die <code>HashMap</code> verwendet die Methoden <code>equals()</code> und <code>hashCode()</code> der Schlüsselobjekte zur Bestimmung der Gleichheit und zum Speichern der Werte.	Wenn benutzerdefinierte Objekte als Schlüssel verwendet werden, die <code>equals()</code> und <code>hashCode()</code> korrekt überschreiben.

Zusammenfassung

Die Verwendung einer `HashMap` ist besonders sinnvoll, wenn:

- Eine schnelle Zuordnung von Schlüsseln zu Werten erforderlich ist.
- Der Zugriff auf Werte über eindeutige Schlüssel erfolgen muss, mit einer durchschnittlichen $O(1)$ Zeitkomplexität.
- Keine Reihenfolge der Elemente beibehalten werden muss.
- Ein dynamisches Hinzufügen und Entfernen von Elementen basierend auf Schlüsseln notwendig ist.